

MEA 1D-CNN Training Pipeline (CTRL vs SCA1)

Line-by-line functional walkthrough and glossary (English).

Date: January 29, 2026

Scope: This document explains what each section of the provided Python script does, in the same order as it appears. It focuses on intent, data flow, and how outputs are produced.

Glossary

Term	Meaning
MEA (Micro-Electrode Array)	A device that records extracellular electrical activity from many electrodes at once, often arranged by wells on a multi-well plate.
Well / Well location (row, col)	A single culture compartment on the plate (e.g., a 48-well plate has 6 rows x 8 columns). The code treats each well as one classification unit.
Electrode	A sensor channel inside a well that records voltage over time. Each well contains multiple electrodes.
Sampling rate (Hz)	How many samples per second are recorded (e.g., 20,000 Hz means 20,000 data points per second).
Downsampling / Resampling	Reducing the sampling rate (e.g., 20 kHz to 4 kHz) to save compute and memory while keeping the main signal content.
Segment / Window	A fixed-length slice of the time series used as one model input example (here defined by <code>seq_len_ms</code>).
1D-CNN	A convolutional neural network operating on 1D sequences (time series). It learns local patterns along the time axis.
Conv1D	A convolution layer that applies filters along time to detect local temporal features.
SeparableConv1D	A more efficient convolution that splits filtering into depthwise (per-channel) + pointwise (channel-mixing) operations.
Depthwise vs Pointwise convolution	Depthwise: filters each channel separately; pointwise (1x1): mixes channels to combine information across electrodes/features.
Dilation (dilation_rate)	Spreads filter taps apart to enlarge the receptive field, allowing the model to capture longer-range temporal patterns without increasing kernel size.
Batch Normalization	Normalizes intermediate activations during training to stabilize gradients and speed up convergence.
ReLU	Non-linear activation function that keeps positive values and zeroes negatives (introduces non-linearity).

MaxPooling1D	Downsamples the sequence by taking maxima in small regions, reducing time resolution and computation.
Dropout	Randomly zeros a fraction of activations during training to reduce overfitting and improve generalization.
GlobalAveragePooling1D	Averages each feature map across time to convert a sequence into a fixed-size vector.
Dense layer	Fully connected layer that mixes learned features to make decisions.
Softmax	Converts final logits into class probabilities that sum to 1 across classes.
Loss (sparse categorical cross-entropy)	Measures how far predicted probabilities are from the true class (integer labels). Lower is better.
Optimizer (Adam)	An adaptive gradient method that updates weights using estimates of first and second moments of gradients.
Learning rate	Step size for weight updates. Too high can diverge; too low can learn very slowly.
Gradient clipping (clipnorm)	Limits gradient norm to avoid unstable large updates.
Train / Validation / Test	Train: used to fit parameters; Validation: used for model selection and early stopping; Test: held out for final evaluation.
Early stopping	Stops training when validation performance stops improving for a set patience.
Class weights	Reweights the loss per class to compensate for imbalanced class frequencies.
Stratified split	Splits data while preserving class proportions (here applied to wells via their labels).
ROC curve	Plots true positive rate vs false positive rate across thresholds for a binary classifier.
AUC	Area under the ROC curve; summarizes ROC performance (1 is perfect, 0.5 is chance).
Confusion matrix	Table of counts comparing true vs predicted labels to visualize errors.
Memmap (memory-mapped array)	Stores arrays on disk but accesses them like NumPy arrays, reducing RAM usage.
tf.data pipeline	TensorFlow input pipeline for streaming batches efficiently during training/evaluation.

Seed / Reproducibility	Fixed random seeds make file selection and random sampling deterministic across runs.
-------------------------------	---

Walkthrough of the Script

The walkthrough follows the script in order. Each section explains **purpose**, **inputs**, **outputs**, and **key implementation details**.

1. Header comments and imports

The opening comment block describes the overall pipeline: a round-based workflow that trains a 1D-CNN on MEA recordings to classify wells as CTRL or SCA1, with optional electrode masking and optional memmap caching.

Imports are grouped by purpose:

- Standard library: file paths, JSON/CSV handling, garbage collection, and filesystem utilities.
- NumPy/h5py: numerical work and reading raw signals from HDF5 recordings.
- Matplotlib/Seaborn: saving plots (training curves, confusion matrices, ROC curves). The backend is set to **Agg** to allow saving figures without a GUI.
- scikit-learn: splitting wells into train/validation, computing metrics, and class weights.
- TensorFlow/Keras: building and training the neural network, plus callbacks for checkpoints and early stopping.

The script prints available TensorFlow devices to confirm whether GPU/CPU is visible in the runtime environment.

2. Config dataclass (experiment parameters)

The **Config** dataclass is the single place where all experiment settings are defined. This makes runs reproducible and easy to compare. Values cover:

- Input sources: manifest path and optional spike-count CSV for electrode masking.
- Plate geometry: number of rows/columns and how channels are reshaped into (rows, cols, electrodes, time).
- Signal settings: raw sampling rate, optional downsampling, segment length in milliseconds.
- Segmentation: number of segments per well for train and validation; stride for evaluation.
- Preprocessing: per-electrode robust normalization (median/MAD), and clipping limits.
- Training: batch size, learning rate, max epochs, early-stopping patience.
- Regularization options: channel shuffle, dropout in the model, and L2 regularization.
- Round workflow: how many files per round for train/val and test, minimum files needed, global test fraction, maximum rounds.
- Performance and memory: memmap caching, tf.data shuffle buffer, prediction chunk size.

After **cfg = Config()**, the script creates the output folder, sets NumPy and TensorFlow seeds, and defines key file paths used throughout the run (global split JSON, used-trainval log, and a global best-model path).

3. Global aggregation dictionary (AGG)

The **AGG** dictionary collects evaluation outputs across rounds so the script can write an overall report at the end.

```
AGG = {
    "y_true": [],          # true labels per evaluated well (across rounds)
    "y_pred": [],          # predicted labels per evaluated well
    "score_vote": [],      # per-well score for the positive class (vote fraction)
)
"classes": None,          # class order used during training/evaluation
```

```
"round_summaries": []# list of per-round metric dictionaries
}
```

4. Spike database loading and electrode masking

load_spike_database reads a CSV where each column corresponds to a measurement/recording ID and each row contains a (well, electrode) spike count. The function builds a lookup:

```
spike_db[(measurement_id, well_id, electrode_id)] = spike_count
```

If the CSV is missing, the script continues without masking and prints a warning.

mask_noisy_electrodes_single uses the lookup to convert spike counts to spike rates (Hz) by dividing by recording duration. For any electrode exceeding **cfg.spike_cutoff_hz**, the entire electrode trace is set to zero, effectively removing it from the input.

Important detail: masking happens on the reshaped array of shape (rows, cols, electrodes, time). A masked electrode becomes all zeros for that well.

5. Manifest loading and bookkeeping for round sampling

load_manifest loads the JSON manifest that lists all recordings. For each record it resolves the file path to an absolute path (**_abs_path**) and filters out missing files.

load_used_trainval_ids and **save_used_trainval_ids** implement persistent bookkeeping to avoid selecting the same train/validation recordings in multiple rounds. The IDs are stored in **used_trainval_ids.json** inside the output folder.

6. Global split (fixed final-test pool)

build_or_load_global_split creates or loads a persistent split of recording IDs:

- A **final test pool** (fraction = **cfg.final_test_fraction**) that is never used for training.
- A **train/val pool** used for round-based training and validation sampling.

If the split file exists, it is loaded so the same final-test IDs are reused across runs. Otherwise, it is created with a seeded shuffle and saved to JSON.

7. Per-round file selection

pick_next_train_files selects the next train/validation files from the train/val pool, excluding IDs already used. It enforces a minimum of **cfg.min_trainval_files**; if fewer remain, the pipeline stops.

pick_test_files_from_final samples a small number of test files from the global final-test pool. This keeps per-round evaluation independent from training.

8. Label assignment from plate layout

The dataset encodes labels by plate column index, defined in **build_label_map_for_columns**.

```
Column indices (0-based):
SCA1: 0, 1, 4, 5
CTRL: 2, 3, 6, 7
```

get_allowed_cols_per_label returns which columns are allowed for each label and optionally restricts CTRL wells to one side (left columns 2-3 or right columns 6-7). This supports experiments where only one CTRL line should be used.

9. Raw data loading, reshaping, downsampling, normalization

load_raw_as_rcte reads the HDF5 dataset at **cfg.key_raw**, producing an array shaped (channels, time). It then:

- Converts to float64 for safe preprocessing, replaces NaN/Inf with 0, and clips extreme values using `cfg.raw_clip_abs`.
- Reshapes into (rows, cols, electrodes, time) based on `cfg.channel_layout` and plate dimensions.
- Returns float32 to reduce memory.

maybe_downsample optionally resamples the time axis to `cfg.target_hz` using polyphase filtering (`scipy.signal.resample_poly`). The function computes a rational ratio p/q via `gcd(fs, target)` to do exact-rate conversion.

normalize_per_electrode applies robust scaling per electrode trace:

```
median = median(x)
MAD = median(|x - median|)
x_norm = (x - median) / (1.4826 * max(MAD, mad_floor) + eps)
then clip to [-norm_clip_abs, +norm_clip_abs]
```

This normalization is robust to outliers and makes training numerically more stable. Clipping prevents occasional extreme normalized values from dominating learning.

10. Building the list of wells (training units)

wells_from_file creates a list of tuples (`file_idx`, `row`, `col`, `label`) for every well that has a known label (CTRL or SCA1). The label is derived from the column index.

filter_wells applies allowed-column constraints (e.g., only one CTRL line).

11. process_file_list

process_file_list loads and preprocesses each recording in a list:

- Loads raw data with `load_raw_as_rcte`.
- Skips files that fail to load or contain non-finite values.
- Optionally masks noisy electrodes using the spike database.
- Optionally downsamples the signal.
- Optionally normalizes per electrode.
- Ensures all values are finite and appends the processed array to a list.

It returns (`processed_list`, `final_fs`). The returned sampling rate is the downsampled rate if downsampling is enabled, otherwise the original `cfg.sampling_hz`.

12. make_segments_to_memmap (disk cache for training samples)

Training uses many short windows per well. To avoid holding all segments in RAM, the script writes them to disk using NumPy memmap.

Inputs:

- `raw_list`: list of preprocessed arrays, each shaped (rows, cols, electrodes, time).
- `wells`: list of well tuples to sample from.
- `n_seg`: segments per well.
- `seq_len`: number of samples per segment (computed from `ms` and sampling rate).
- `out_x_path/out_y_path`: output memmap file paths.
- `name2idx`: optional mapping from label string to integer class index (reused across train/val).
- `shuffle_channels`: optional electrode channel shuffling (regularization).

Key steps inside the function:

- Build an **eligible** list of wells: correct electrode count, not fully masked (not all zeros), and long enough for `seq_len`.

- Define or reuse the class-to-index mapping (name2idx).
- Allocate memmaps: Xmm with shape (N, seq_len, E) and ymm with shape (N,).
- For each eligible well: randomly sample start positions and extract segments.
- Each segment is stored as (time, electrodes) by transposing the per-well signal slice.
- Optional channel shuffling: permute electrodes to reduce reliance on fixed electrode order.
- Flush and close memmaps, return N, classes, name2idx, and x_shape.

Output convention: The model input expects shape (seq_len, n_chan). Here n_chan corresponds to the number of electrodes per well, and seq_len is the number of time samples in the window.

13. MemmapBatchLoader and dataset_from_memmap

MemmapBatchLoader lazily opens the memmap files and provides **read_batch** to retrieve indexed batches. It also replaces NaN/Inf to avoid TensorFlow instability.

dataset_from_memmap builds a tf.data.Dataset that:

- Creates an index dataset of 0..N-1.
- Optionally shuffles indices for training (buffer size capped by cfg.tf_shuffle_buffer).
- Batches indices, then maps them to actual (X, y) batches via tf.py_function calling the loader.
- Prefetches 1 batch to overlap I/O and compute.
- For reproducibility, sets experimental_deterministic=True.
- Attaches the loader object to the dataset so it can be closed during cleanup.

14. build_model (1D-CNN architecture)

The model takes an input tensor shaped (seq_len, n_chan) where n_chan is the number of electrodes per well, and outputs a softmax probability over classes.

Layer-by-layer structure:

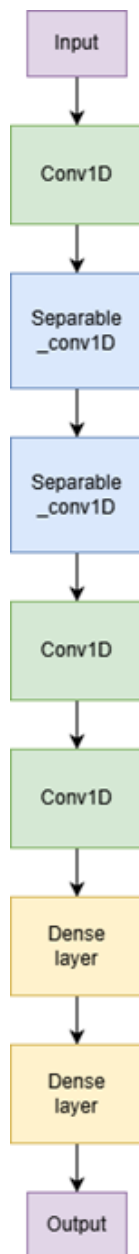


Figure: CNN architecture flow used in this script.

```

Input: (seq_len, n_chan)
Conv1D(filters=16, kernel=1) + BatchNorm + ReLU
SeparableConv1D(filters=16, kernel=5) + BatchNorm + ReLU + MaxPool(2)
SeparableConv1D(filters=32, kernel=5) + BatchNorm + ReLU + MaxPool(2)
Conv1D(filters=32, kernel=5, dilation=1) + BatchNorm + ReLU + MaxPool(2)
Conv1D(filters=32, kernel=5, dilation=2) + BatchNorm + ReLU + MaxPool(2)
Dropout(0.5)
GlobalAveragePooling1D
Dense(16, ReLU, L2 regularization)
Dropout(0.5)
Dense(n_classes, Softmax)
  
```

The model is compiled with Adam(learning_rate=cfg.lr, clipnorm=1.0), sparse categorical cross-entropy loss, and accuracy metric.

Regularization is applied through dropout, L2 penalties in convolution and dense layers, and (optionally) channel shuffling during training segment creation.

15. plot_curves, ROC utilities, and plot_roc_binary

plot_curves saves loss and accuracy curves for train and validation from the Keras History object. Figures are written into the round folder.

_sanitize_binary_roc_inputs ensures `y_score` values are finite, clipped to `[0, 1]`, and aligned with `y_true`. This prevents runtime errors.

plot_roc_binary computes ROC and AUC for the positive class, saves a ROC curve plot and a text file with the numeric AUC.

16. consensus_eval_multifile (well-level evaluation by vote)

Evaluation is performed at the **well** level, even though the model predicts on **segments**.

Core idea: each well is split into many sliding windows; the model predicts a class per window; then predictions are aggregated into one well-level decision.

Detailed flow:

- Compute `seq_len` (samples) from `seq_len_ms` and `fs`; compute stride from `stride_ms`.
- Determine the positive class index (`pos_idx`) within `classes_train` (must include `cfg.pos_label`).
- For each test file and each labeled well: build window start indices from `0..T-seq_len` with `step=stride`.
- Predict probabilities in chunks (`cfg.eval_pred_chunk`) to reduce peak memory.
- Convert each window prediction to a hard class (`argmax`) and count votes per class.
- Well prediction is the class with the highest vote count.
- Well `score_pos` is the fraction of windows voting for the positive class (`vote_pos = counts[pos_idx]/total`).
- Acc/weighted precision/weighted F1 are computed from well-level `y_true/y_pred`.
- A confusion matrix heatmap is saved, plus a per-well CSV and a text report.
- ROC/AUC is computed using `vote_pos` as the continuous score for the positive class.

If model predictions contain non-finite values for a well, that well is skipped to keep metrics reliable.

17. write_round_summaries_csv and write_overall_metrics

write_round_summaries_csv writes one line per round: `n_wells_eval`, `acc`, weighted precision, weighted F1, and AUC (if available).

write_overall_metrics aggregates all evaluated wells across rounds (from AGG) and computes overall metrics and plots. It also writes an `overall_summary.json` containing key run metadata (paths, number of rounds, positive label, and whether memmap caching was used).

18. Cleanup helpers

_safe_remove deletes cache files without crashing if removal fails. It is used to clean up memmap files after each round finishes.

19. train_one_round (end-to-end round execution)

This is the main orchestration function for a single round. It performs split creation, preprocessing, segmentation, training, evaluation, saving, and cleanup.

Step-by-step:

- Create a round tag and folder (e.g., `round001`) and define checkpoint paths (`best_model.keras`, `last_model.keras`).
- Print file names used for train/val and test in this round for transparency.
- Preprocess train/val recordings and test recordings via `process_file_list`.

- Build a list of wells from all training recordings, then split wells into train and validation using stratified train_test_split (80/20).
- Apply allowed-column filtering to training wells (optional CTRL-line restriction).
- Write training and validation segments to memmap files (Xtr/ytr and Xva/yva).
- Create tf.data datasets that read batches from memmap on CPU.
- Warm start: if a global best model exists, load it; otherwise build a new model.
- Set callbacks: ModelCheckpoint (best on val_accuracy), EarlyStopping (restore best weights), ReduceLROnPlateau.
- Compute class weights from ytr memmap and train with model.fit using class_weight.
- Save the last model and update the global best model by copying the best checkpoint.
- Save history.json and training curves plots.
- Evaluate on the per-round test files using consensus_eval_multifile and store results in AGG and round_summaries.json.
- Always run cleanup: close memmap loaders, clear TensorFlow session, garbage collect, and remove memmap cache files if enabled.

Note on memory: raw training arrays are deleted after writing memmap segments. Training then streams batches from disk, which keeps RAM usage low even with many segments.

20. main() and script execution

main() is the entry point. It validates basic settings, loads the manifest and spike database, prepares the global split, and then iterates over rounds until either:

- The pool is exhausted below the minimum train/val file count, or
- cfg.max_rounds is reached (if non-zero).

After finishing rounds, the script writes overall metrics and a final round_summaries CSV.

The block `if __name__ == '__main__': main()` ensures main runs only when executing the script directly.

Data Flow Summary

This summary ties together how data moves through the pipeline from disk to metrics.

Stage	What happens
Input	manifest.json points to HDF5 recordings; optional spike CSV provides per-electrode spike counts.
Global split	IDs are split once into a train/val pool and a final test pool and saved to JSON.
Round selection	Train/val files are selected from the pool without reuse; test files are sampled from the final pool.
Preprocessing	Each file is reshaped to (rows, cols, electrodes, time), optionally masked, downsampled, and normalized.
Segmentation	For each labeled well: random windows are extracted and written to memmap arrays X (float32) and y (int32).
Training	tf.data streams batches from memmap; model is trained with class weights and early stopping; best checkpoint saved.
Evaluation	Each test well is split into windows; window predictions are aggregated to a well-level vote and score_pos.
Reporting	Per-round reports/plots are saved; aggregated metrics across rounds are computed at the end.